Comandos para la defensa (copiar y pegar)

Antes de todo: Docker Desktop abierto (ballena en la barra, "Engine running"). Todos los comandos se pegan en **PowerShell** y funcionan sin internet, salvo que se diga lo contrario. Cada uno entra a `~/pochoco_asic` en Ubuntu (WSL).

---

1. Regenerar el layout (GDS) con la corrida final

Corre todo el flujo (síntesis, floorplan, placement, reloj, ruteo, DRC, LVS) con T = 100 ns y U = 40 %. Dura unos 5-7 minutos. Reemplaza la corrida `final_opt`.

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && source ~/ll/bin/activate && RUN_TAG=final
_opt scripts/run_final.sh"
```

Termina con `Listo: librelane/runs/final_opt`. El GDS queda en `librelane/runs/final_opt/final/gds/asic_top.gds` (se abre con KLayout de Windows: `File > Open`). `--overwrite` ya viene incluido, así que pisa la corrida anterior con el mismo nombre.

2. Ver el layout en OpenROAD (visor gráfico, desde Docker)

Marca `final_opt` como la última corrida y abre la ventana. Dentro: `Windows` para jerarquía y capas, `Heat Maps` para congestión, clic derecho en una celda para resaltar el camino crítico.

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && touch librelane/runs/final_opt && source
~/ll/bin/activate && python3 -m librelane --dockerized --pdk gf180mcuD --pdk-root ~/.cie
l --last-run --flow OpenInOpenROAD librelane/config.json"
```

Debe decir `Using existing run at '...final_opt'`. Si dice `sim_corto`, repite el comando (el `touch` actualiza la fecha).

3. Ver el layout en KLayout (sin Docker)

Abre KLayout en Windows y `File > Open` sobre `resultados > final_opt > gds > asic_top.gds`. Tecla `F` = ver todo el chip. El layout anotado (banco de registros, ALU, contador) está en `informe/figuras/layout_anotado_klayout.png`.

4. Resumen de métricas de todas las corridas

Escribe `librelane/resumen_corridas.csv` (WNS/TNS, área, costo, DRC, LVS, antenas, potencia).

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && source ~/ll/bin/activate && python3 scrip
ts/collect_metrics.py"
```

5. Regenerar el programa desde game.s

El assembler traduce `sw/game.s` a `sw/game.hex` y a la ROM Verilog (`rtl_asic/gen/`). Después hay que repetir la corrida final (comando 1) para que el chip tenga el programa nuevo.

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && make program"
```

Si cambias `CLOCK_PERIOD`, recalcula las 4 constantes al inicio de `game.s` (comentario de cálculo incluido). A 10 MHz: espera 30.000.000, décima 1.000.000, antirrebote 200.000, parpadeo 2.500.000 ciclos.

6. Pruebas y simulaciones del RTL

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && make test"
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && make sim-isa"
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && make sim"
```

- `make test`: 25 pruebas del assembler (deben decir `OK`).
- `make sim-isa`: instrucciones del Espino sobre `asic_top` (`ISA: OK`).
- `make sim`: partida completa con tiempos cortos (`JUEGO: OK - todas las reglas se cumplen`).
- `make sim-ff`: lo mismo con la estrategia 2 (memoria de flip-flops).

7. Simulación post-síntesis (3.7)

Primero se sintetiza con la ROM de constantes cortas (solo Yosys, ~3 min) y luego se simula esa netlist real de GF180 y se compara con el RTL.

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && source ~/ll/bin/activate && scripts/run_s
im_corto.sh"
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && make sim-post NETLIST=librelane/runs/sim_
corto/06-yosys-synthesis/asic_top.nl.v"
```

Resultado esperado: `JUEGO: OK` y `TRAZAS IGUALES` (66 eventos en el RTL y en la netlist).

8. Barridos (3.4 y 3.5)

Cada punto tarda unos 5-7 minutos.

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && source ~/ll/bin/activate && scripts/sweep
_clock.sh 80 70 60 50"
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && source ~/ll/bin/activate && scripts/sweep
_util.sh 30 50 60"
```

- `sweep_clock.sh`: varía CLOCK_PERIOD (ns). El de 100 ns es la corrida final.
- `sweep_util.sh`: varía FP_CORE_UTIL (%). El de 40 % es la corrida final.

9. Estrategia 2 (memoria de flip-flops)

Se puede estancar en el ruteo (horas): no la lances si vas a cerrar el laptop. Para detener una corrida pegada: `wsl -d Ubuntu -e bash -c "pkill -f librelane"` y luego `docker stop` del contenedor desde Docker Desktop.

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && source ~/ll/bin/activate && MEM=ff RUN_TA
G=final_mem_ff scripts/run_final.sh"
```

10. Versiones de las herramientas

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && scripts/versiones_herramientas.sh"
```

Ya están en el README y en `versiones_herramientas.txt`: LibreLane 3.0.17, OpenROAD build 2026-02-17, Yosys 0.62, Magic 8.3 rev. 623, Netgen 1.5.316, OpenSTA 2.7.0, PDK gf180mcuD (ciel `54435919abffb937387ec956209f9cf5fd2dfbee`), celdas `gf180mcu_fd_sc_mcu7t5v0`.

11. Llevar resultados a Windows y subir al repo

```
wsl -d Ubuntu -e bash -c "cd ~/pochoco_asic && scripts/export_to_windows.sh"
```

Luego, en PowerShell, desde la carpeta del proyecto:

```
cd C:\Users\felip\Downloads\Proyecto_3_Arquitectura\pochoco_asic
git add .
git commit -m "mensaje"
git push
```

Si algo falla

- **`Cannot connect to the Docker daemon`:** abre Docker Desktop, espera a "Engine running" y repite.
- **`PDK gf180mcuD was not found`:** falta `--pdk-root ~/.ciel` (los comandos de arriba ya lo traen).
- **Un comando no hace nada o no existe `~/ll/bin/activate`:** el entorno de Python está en `~/ll`; los comandos de arriba lo activan con `source ~/ll/bin/activate`.
- **Hay que traer archivos nuevos de Windows a Ubuntu:** `cp -u` desde `/mnt/c/Users/felip/Downloads/Proyecto_3_Arquitectura/pochoco_asic/...` hacia `~/pochoco_asic/...`.

Números clave de la corrida final (`final_opt`, T = 100 ns, U = 40 %)

Dato	Valor
Frecuencia de operación	10 MHz (período 100 ns)
Frecuencia máxima	~15,5 MHz (peor corner `max_ss`), ~29 MHz (típico)
Setup WNS / TNS	+37,22 ns / 0 (peor corner)
Hold WNS / TNS	+0,17 ns / 0
DRC Magic / LVS / antenas	0 / 0 / 0
Die / costo	0,4416 mm² / US$10.156 (a US$23.000/mm²)
Celdas tras síntesis / flip-flops	5.567 / 1.069
Módulo de mayor área	banco de registros: 45,3 % (ALU 6,1 %, contador de ciclos 2,4 %)
Camino crítico	`instr_rdata` → decodificador → ALU → ... → registro del banco (escritura), ~65 ns en `max_ss`
Palabras usadas por el juego	114 de instrucciones, 0 de datos
Estrategia 2 (parcial)	4,0x área, 4,4x flip-flops, fmax ~1,19 MHz, ruteo sin cerrar